7. Level 2: Container diagram

Once you understand how your software system fits in to the overall environment with a System Context diagram, a useful next step is to illustrate the high-level technology choices with a Container diagram.

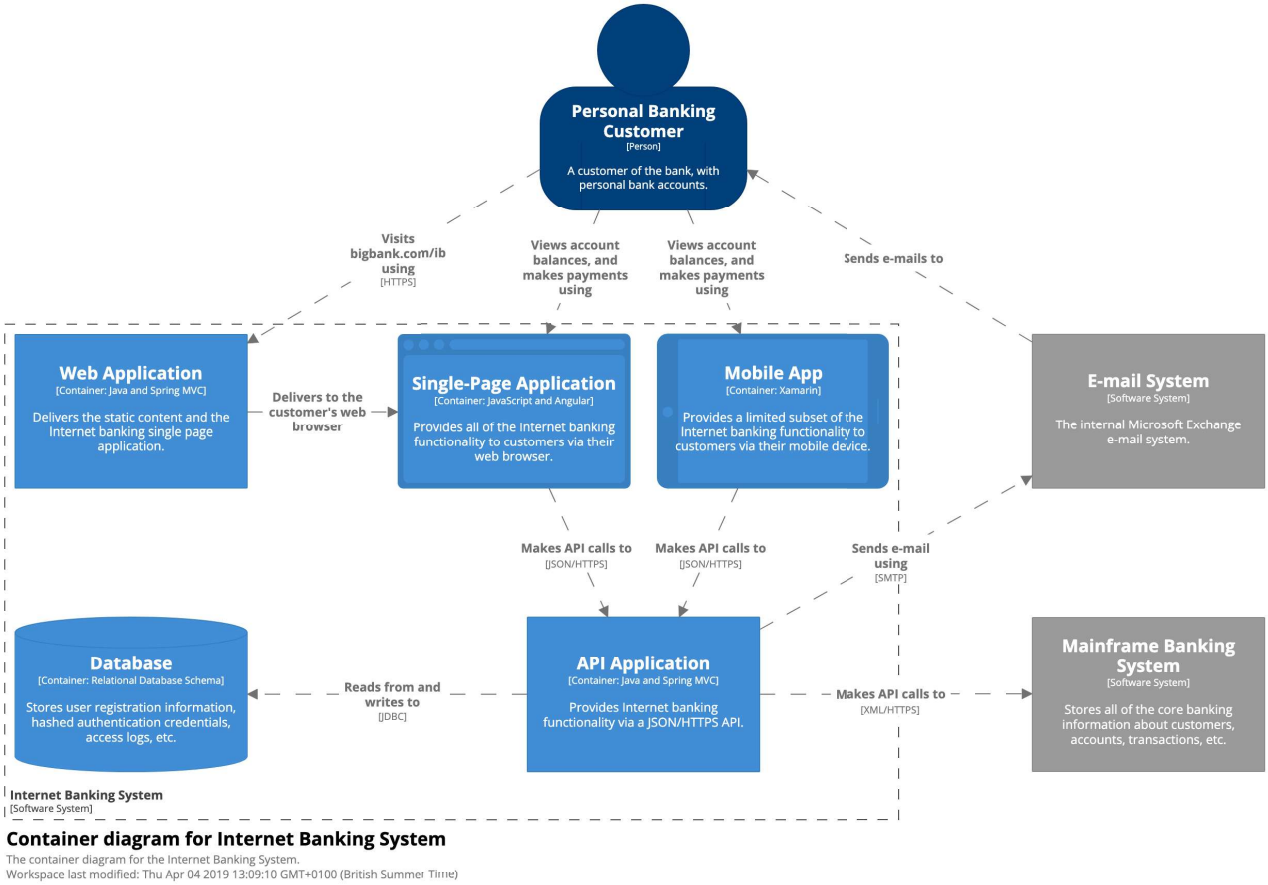
7.1 Intent

The Container diagram shows the high-level shape of the software architecture and how responsibilities are distributed across it. It also shows the major technology choices, how they are used, and how containers communicate with each other. It's a high-level *technology focussed* diagram that is useful for software developers and support/operations staff alike. A container diagram helps you answer the following questions:

1. What is the overall shape of the software system?
2. What are the high-level technology decisions?
3. How are responsibilities distributed across the system?
4. How do containers communicate with one another?
5. As a developer, where do I need to write code in order to implement features?

7.2 Structure

Draw a block diagram showing the high-level technical elements (containers) that your software system consists of. This is an example Container diagram for the fictional Internet Banking System.



A Container diagram for the Internet Banking System

It shows that the Internet Banking System (the dashed box) is made up of five containers: a server-side Web Application, a Single-Page Application, a Mobile App, a server-side API Application, and a Database. The Web Application is a Java/Spring MVC web application that serves static content (HTML, CSS and JavaScript), including the content that makes up the Single-Page Application. The Single-Page Application is an Angular application that runs in the customer's web browser, providing all of the Internet banking features. Alternatively, customers can use the cross-platform Xamarin Mobile App, to access a subset of the Internet banking functionality.

Both the Single-Page Application and Mobile App use a JSON/HTTPS API, which is provided by another Java/Spring MVC application running on the server. The API Application gets user information from the Database (a relational database schema). The API Application also communicates with the existing Mainframe Banking System, using a proprietary XML/HTTPS interface, to get information about bank accounts or make transactions. The API Application also uses the existing E-mail System if it needs to send e-mails to customers.

It's worth pointing out that this diagram says nothing about the number of physical instances

of each container. For example, the API Application could be running on a farm of Apache Tomcat servers, with the relational database running on an Oracle cluster, but this diagram doesn't show that level of information. Instead, I show physical instances, failover, clustering, etc on a separate Deployment diagram that illustrates the mapping of containers onto infrastructure.

How much detail?

If you're drawing a Container diagram during an up-front design exercise, you might not have some of the technical details to hand. That's fine, just add what you know. If, on the other hand, you're drawing a diagram to document an existing system, it's more likely that you'll be able to add some of the finer details; such as protocols, port numbers, etc. The choice is yours, add as much detail as you feel is necessary.

7.3 Elements

A container diagram usually includes three types of elements; people, software systems and containers.

Containers

A container typically represents an application or data store. I'll capture the following information about each container:

- **Name**: The name of the container (e.g. "Internet-facing web server", "Database", etc).
- **Technology**: The implementation technology (e.g. Java/Spring MVC application, ASP.NET web application, C# Windows Service, Relational Database Schema, etc).
- **Description**: A short descriptive statement, or perhaps a list of the container's key responsibilities or entities/tables/files/etc that are being stored.

If you're undertaking an upfront design exercise and you don't quite know which technology you're going to use to build a particular container, then perhaps instead mention the short-list of technology choices or the general type of technology (e.g. "Relational database schema"). On that note, if a container is a standard RDBMS database/schema, and doesn't make use of any proprietary features, you don't necessarily need to specify the technology. Instead you could mention the technology on a deployment diagram. But if your database is using

specific features of the RDBMS (e.g. Oracle stored procedures), and can't be deployed onto a different RDBMS without changes, then you should specify the technology to make that clear.

File systems and log files vs data storage

If I'm describing a software system where an application stores business critical data on a file system, I will include a "File System" container on the diagram, to make this explicit. In contrast, although many types of containers will write log files to a file system (and this is undoubtedly important), I typically omit this detail for brevity.

Data storage as a service

A frequently asked question is whether services like Amazon S3, Amazon RDS, or Azure SQL Database should be shown on a container diagram. After all, these are external services that we don't own or run ourselves.

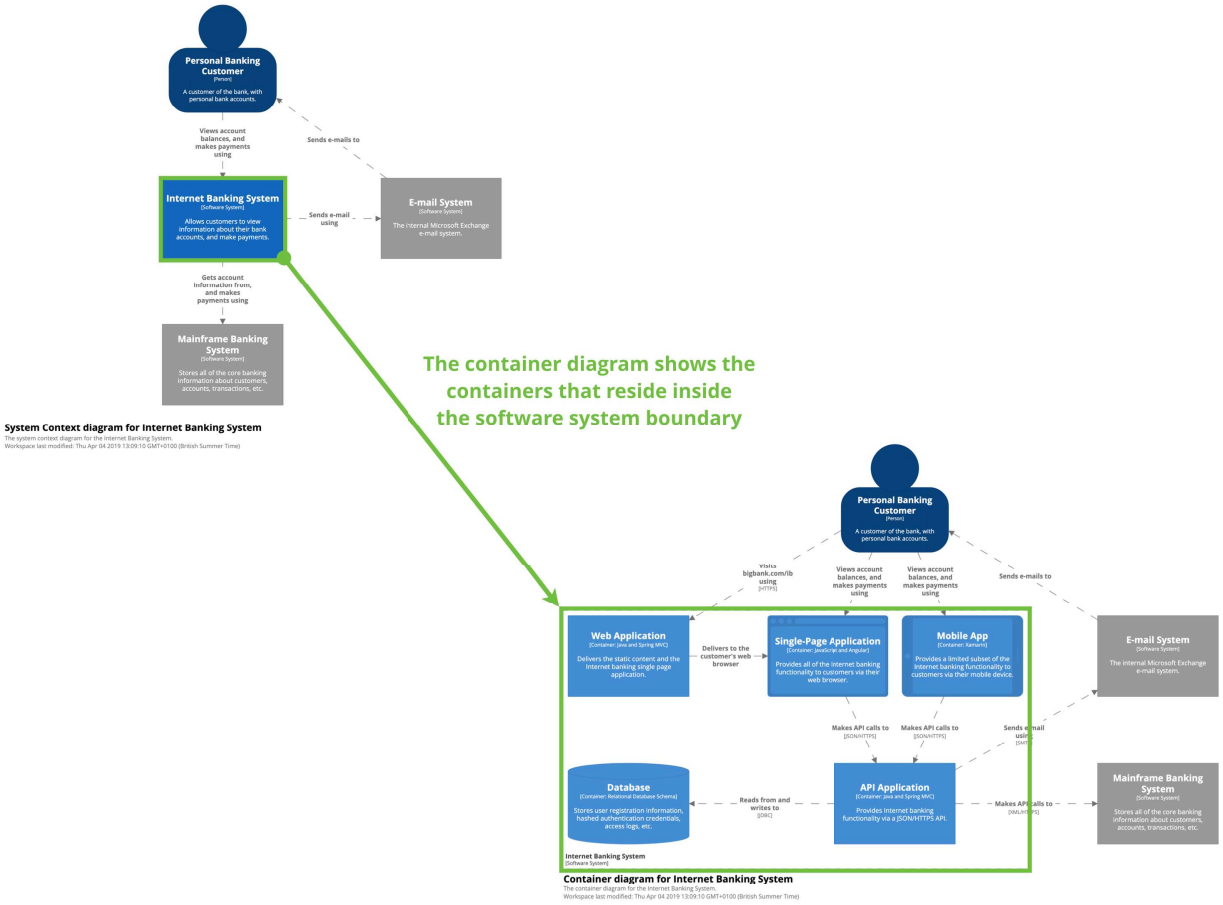
If you're building a software system that is using Amazon S3 for storing data, it's true that you don't run S3 yourself, but you do have ownership and responsbility for the buckets you are using. Similarly with Amazon RDS, you have (more or less) complete control over any database schemas that you create. For this reason, I *will* usually show such containers on a container diagram. They are an integral part of your software architecture, although they are hosted elsewhere.

People and software systems

If you look back at the example Container diagram, you'll notice that it has the same people and software systems as the example System Context diagram. I usually include the same collection of people (users, actors, personas, etc) and software systems because it helps to tell the overall story of how the system works and it provides continuity between the two diagrams. In effect, a Container diagram is very similar to the System Context diagram, where you're zooming in to see what's inside the software system in scope.

Software system boundary

With this in mind, I recommend drawing a bounding box around the containers on your diagram to explicitly show the system boundary. This system boundary should correspond with the single box that appears on the System Context diagram.



The Container diagram shows a zoom in of the software system boundary

7.4 Interactions

Typically, communication between containers is *out-of-process* (or *inter-process*). It's very useful to explicitly identify this and summarise how these interactions will work. As with any diagram, I recommend annotating all interactions rather than having a diagram with a collection of boxes and ambiguous unlabelled lines connecting everything together. Useful information to annotate the interactions with includes:

- The purpose of the interaction (e.g. "reads/writes data from", "sends reports to", etc).
- The communication mechanism (e.g. Web Services, REST, Web API, Java Remote Method Invocation, Windows Communication Foundation, Java Message Service).
- The communication style (e.g. synchronous, asynchronous, batched, two-phase commit, etc).

- Protocols and port numbers (e.g. HTTP, HTTPS, SOAP/HTTP, SMTP, FTP, RMI/IIOP, etc).

7.5 Motivation

Where a System Context diagram shows your software system as a single box, a Container diagram opens this box up to show what's inside it. This is useful because:

- It makes the high-level technology choices explicit.
- It shows the relationships between containers, and how those containers communicate.

During my software architecture sketching workshops, I often see groups producing a high-level context diagram that shows the software system in question as a single black box, and a second diagram that shows *all* of the components that reside within the entirety of the software system. This second diagram is usually cluttered and confusing because it presents too much information. It's also usually not clear how the components are grouped together from a deployment perspective. The Container diagram provides a nice intermediate diagram between the two contrasting levels of abstraction. It also logically leads you to level 3 (a Component diagram), where you open up a single container to show only the components that reside within that container.

7.6 Audience

Technical people inside and outside of the immediate software development team; including everybody from software developers through to operational and support staff.

7.7 Required or optional?

All software systems should have a Container diagram.